

**LAPORAN PRAKTIKUM
MACHINE LEARNING (PRAKTIKUM)**



Nama : Shendy Tria Amelyana
NIM : 434231003
Kelas : C1

DOSEN PENGAMPU :
INDAH WERDININGSIH, S.SI., M.Kom.

**PROGRAM STUDI DIPLOMA IV TEKNIK INFORMATIKA
FAKULTAS VOKASI
UNIVERSITAS AIRLANGGA
SURABAYA
2025**

BAB 1

PENDAHULUAN

1.1 Latar Belakang

Dalam era digital saat ini, pengolahan data secara otomatis menjadi kebutuhan penting di berbagai bidang. Salah satu metode yang sering digunakan dalam *machine learning* untuk klasifikasi adalah Naive Bayes Classifier.

Algoritma ini bekerja dengan prinsip teorema Bayes, yaitu menghitung peluang suatu kelas berdasarkan nilai fitur yang dimiliki suatu data.

Namun, proses perhitungan Naive Bayes sering kali dianggap kompleks jika dilakukan secara manual. Oleh karena itu, pada praktikum ini dibuat sebuah aplikasi berbasis web menggunakan Flask (Python) untuk mempermudah proses perhitungan Naive Bayes secara otomatis, menampilkan hasil analisis, dan memberikan visualisasi seperti *confusion matrix* serta akurasi model.

1.2 Tujuan

Tujuan dari praktikum ini adalah:

1. Menerapkan algoritma Naive Bayes untuk proses klasifikasi data.
2. Mengimplementasikan perhitungan prior probability, likelihood, dan frequency table pada dataset.
3. Membuat aplikasi web sederhana menggunakan Flask untuk menganalisis dataset CSV.
4. Menampilkan hasil analisis berupa tabel, akurasi, *classification report*, dan *confusion matrix*.
5. Memungkinkan pengguna melakukan prediksi manual melalui web interface.

BAB II

PEMBAHASAN

2.1 Dasar Teori

a. Teorema Bayes

Teorema Bayes digunakan untuk menghitung probabilitas suatu hipotesis berdasarkan informasi atau bukti yang tersedia:

$$P(H | E) = \frac{P(E | H) \times P(H)}{P(E)}$$

Keterangan:

- $P(H | E)$ = probabilitas hipotesis H benar jika diketahui evidence E (posterior)
- $P(H)$ = probabilitas awal dari hipotesis H (prior probability)
- $P(E | H)$ = probabilitas evidence E jika hipotesis H benar (likelihood)
- $P(E)$ = probabilitas evidence E (konstanta)

b. Probabilitas Prior

Adalah peluang dasar suatu kelas sebelum melihat fitur.

Contoh: jika dari 100 data, 60 berlabel “Ya” dan 40 “Tidak”, maka:

- $P(Ya) = 0.6$
- $P(Tidak) = 0.4$

c. Likelihood

Merupakan peluang suatu fitur muncul berdasarkan kelas tertentu.

Contoh: $P(Fitur = A | Kelas = Ya)$ adalah peluang fitur “A” muncul jika diketahui kelasnya “Ya”.

d. Posterior Probability

Adalah hasil akhir dari perhitungan Naive Bayes, yaitu peluang terbesar yang menentukan kelas prediksi suatu data baru.

2.2 Alat dan Bahan

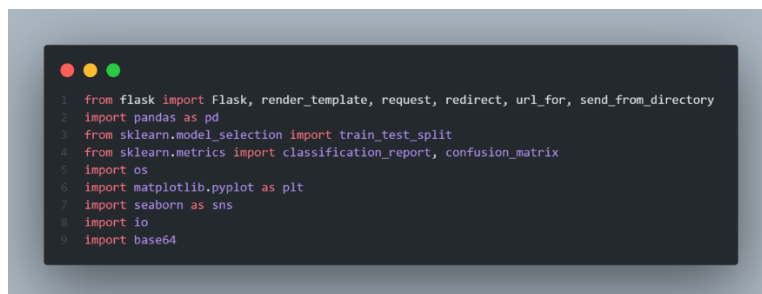
- Bahasa Pemrograman: Python 3
- Framework: Flask
- Library: pandas, scikit-learn, matplotlib, seaborn
- Text Editor: Visual Studio Code
- Browser: Microsoft Edge

- Dataset: file CSV (unggahan pengguna)

2.3 Langkah Kerja

1. Mengunggah dataset CSV melalui halaman utama aplikasi Flask.
2. Program membaca dataset menggunakan pandas dan memisahkan fitur serta target.
3. Data dibagi menjadi 80% data training dan 20% data testing menggunakan `train_test_split`.
4. Sistem menghitung:
 - Probabilitas prior setiap kelas.
 - Tabel frekuensi untuk tiap fitur terhadap target.
 - Likelihood dari setiap kombinasi nilai fitur dan kelas.
5. Melakukan prediksi Naive Bayes terhadap data uji.
6. Menghitung akurasi, classification report, dan menampilkan confusion matrix dalam bentuk visual.
7. Menyediakan fitur prediksi manual dengan input langsung dari pengguna.
8. Menyimpan seluruh hasil analisis (tabel, grafik, prediksi) dalam format CSV yang dapat diunduh.

2.4 Cuplikan Kode Penting



```

1 from flask import Flask, render_template, request, redirect, url_for, send_from_directory
2 import pandas as pd
3 from sklearn.model_selection import train_test_split
4 from sklearn.metrics import classification_report, confusion_matrix
5 import os
6 import matplotlib.pyplot as plt
7 import seaborn as sns
8 import io
9 import base64

```

- Flask: membuat web app, upload file, dan menampilkan halaman HTML.
- Pandas: membaca dan mengolah file CSV.
- scikit-learn: fungsi `train_test_split`, `classification_report`, dan `confusion_matrix`.
- matplotlib & seaborn: untuk visualisasi *confusion matrix*.
- os, io, base64: mengelola file dan menyimpan gambar dalam format HTML (base64).



```

1 app = Flask(__name__)
2 app.config["UPLOAD_FOLDER"] = "uploads"
3 os.makedirs(app.config["UPLOAD_FOLDER"], exist_ok=True)

```

Membuat objek Flask dan menentukan folder tempat menyimpan file upload (uploads/).

```
1 def ensure_dir(path: str):
2     os.makedirs(path, exist_ok=True)
```

Membuat folder jika belum ada.

```
1 def save_table(df: pd.DataFrame, path: str, index: bool = False):
2     df.to_csv(path, index=index, encoding="utf-8-sig")
```

Menyimpan DataFrame ke file CSV.

```
1 def build_export_links(base_dir, scope_name, features, freq_tables_clean, like_tables_raw, prior_df):
2     """
3     - freq_tables_clean: sudah reset_index (kolom 1 = nama fitur) -> simpan index=False
4     - like_tables_raw: DataFrame likelihood dengan index nilai + 'All'
5     -> kita reset_index agar kolom 1 = nama fitur lalu simpan index=False
6     """
7     scope_dir = os.path.join(base_dir, scope_name)
8     ensure_dir(scope_dir)
9     links = {"prior": None, "freq": [], "like": []}
10
11     # prior
12     prior_csv = os.path.join(scope_dir, f"prior_{scope_name}.csv")
13     prior_df.to_csv(prior_csv, index=False, encoding="utf-8-sig")
14     rel_prior = os.path.relpath(prior_csv, os.path.join(app.config["UPLOAD_FOLDER"], "exports"))
15     links["prior"] = url_for("download_export", subpath=rel_prior.replace("\\", "/"))
16
17     # freq & like per fitur
18     for feat in features:
19         # Frequency (sudah bersih)
20         freq_csv = os.path.join(scope_dir, f"frequency_{feat}.csv")
21         freq_tables_clean[feat].to_csv(freq_csv, index=False, encoding="utf-8-sig")
22         rel_freq = os.path.relpath(freq_csv, os.path.join(app.config["UPLOAD_FOLDER"], "exports"))
23         links["freq"].append((feat, url_for("download_export", subpath=rel_freq.replace("\\", "/"))))
24
25         # Likelihood (reset_index agar header kiri = nama fitur)
26         like_df_raw = like_tables_raw[feat]
27         feat_name = like_df_raw.index.name or feat
28         like_clean = like_df_raw.reset_index().rename(columns={feat_name: feat})
29         like_csv = os.path.join(scope_dir, f"likelihood_{feat}.csv")
30         like_clean.to_csv(like_csv, index=False, encoding="utf-8-sig")
31         rel_like = os.path.relpath(like_csv, os.path.join(app.config["UPLOAD_FOLDER"], "exports"))
32         links["like"].append((feat, url_for("download_export", subpath=rel_like.replace("\\", "/"))))
33     return links
```

Menyimpan hasil perhitungan Naive Bayes (*prior*, *frequency*, *likelihood*) ke folder dan membuat link unduh untuk ditampilkan di web.

```

1 def build_split_links(base_dir, x_train, y_train, x_test, y_test, y_pred, target_col: str):
2     split_dir = os.path.join(base_dir, "splits")
3     ensure_dir(split_dir)
4
5     y_train_df = y_train.to_frame(name=target_col)
6     y_test_df = y_test.to_frame(name=target_col)
7     y_pred_df = pd.DataFrame({"Predicted": y_pred})
8
9     files = {
10         "x_train": ("x_train.csv", x_train, False),
11         "y_train": ("y_train.csv", y_train_df, False),
12         "x_test": ("x_test.csv", x_test, False),
13         "y_test": ("y_test.csv", y_test_df, False),
14         "y_pred": ("y_pred.csv", y_pred_df, False),
15     }
16     links = {}
17     for key, (fname, df, use_index) in files.items():
18         fpath = os.path.join(split_dir, fname)
19         save_table(df, fpath, index=use_index)
20         rel = os.path.relpath(fpath, os.path.join(app.config["UPLOAD_FOLDER"], "exports"))
21         links[key] = url_for("download_export", subpath=rel.replace("\\", "/"))
22     return links

```

Menyimpan hasil *split* data dan hasil prediksi ke file CSV agar bisa diunduh.

```

1 @app.route("/", methods=["GET", "POST"])
2 def index():
3     if request.method == "POST":
4         file = request.files.get("csv_file")
5         if not file or file.filename == "":
6             return render_template("index.html", error="Tidak ada file yang diupload.")
7         file_path = os.path.join(app.config["UPLOAD_FOLDER"], file.filename)
8         file.save(file_path)
9         return redirect(url_for("analyze", filename=file.filename))
10    return render_template("index.html")

```

- Menampilkan halaman upload (index.html).
- Jika file diunggah, disimpan ke folder uploads/ lalu diarahkan ke route /analyze/<filename>.

```

1 df = pd.read_csv(file_path)
2
3 if df.isnull().any().any():
4     return render_template("result.html", error="Dataset mengandung nilai kosong.")
5
6 # Pisahkan fitur & target
7 target_col = df.columns[-1]
8 features = df.columns[:-1]
9 X_full = df[features].astype(str)
10 y_full = df[target_col].astype(str)

```

- Dataset dibaca dari file CSV.
- Kolom terakhir dianggap sebagai *target* (kelas).
- Sisanya dianggap *fitur*.

```

1 X_train, X_test, y_train, y_test = train_test_split(
2     X_full, y_full, test_size=0.2, random_state=42, stratify=y_full
3 )

```

Data dibagi 80% untuk training dan 20% untuk testing secara stratifikasi (proporsi kelas seimbang).

```

1 class_counts_train = y_train.value_counts()
2 priors_train = class_counts_train / class_counts_train.sum()

```

Prior = peluang munculnya setiap kelas.

```

1 for col in features:
2     freq_raw = pd.crosstab(
3         train_df[col], train_df[target_col],
4         margins=True, margins_name="Grand Total"
5     )

```

Membuat tabel frekuensi nilai setiap fitur terhadap setiap kelas target.

```

1 for col in features:
2     freq = pd.crosstab(train_df[col], train_df[target_col])
3     like = freq.div(class_counts_train, axis=1)

```

Likelihood = peluang nilai fitur *jika diketahui kelas*.

```

1 def predict_row(row_dict):
2     posteriors = {}
3     for cls in class_counts_train.index:
4         prob = float(priors_train[cls])
5         for col, val in row_dict.items():
6             val = str(val).strip()
7             like_tab = likelihoods_train_raw[col]
8             if val in like_tab.index:
9                 prob *= float(like_tab.loc[val, cls])
10            else:
11                prob *= 1e-9
12            posteriors[cls] = prob
13        total = sum(posteriors.values())
14        if total > 0:
15            for cls in posteriors:
16                posteriors[cls] /= total
17        return max(posteriors, key=posteriors.get)

```

- Mengalikan prior dan semua likelihood untuk tiap kelas.
- Memilih kelas dengan nilai probabilitas terbesar sebagai hasil prediksi.

```

1 y_pred = [predict_row(X_test.iloc[i].to_dict()) for i in range(len(X_test))]
2
3 accuracy = round((y_test.values == pd.Series(y_pred).values).mean() * 100, 2)
4 report = classification_report(y_test, y_pred, output_dict=False)
5
6 # Confusion matrix
7 labels = sorted(y_full.unique().tolist())
8 cm = confusion_matrix(y_test, y_pred, labels=labels)

```

- Melakukan prediksi pada data uji.
- Menghitung akurasi, laporan klasifikasi, dan *confusion matrix*.

```

1 sns.heatmap(cm, annot=True, fmt="d", cmap="PuBuGn", xticklabels=labels, yticklabels=labels)
2 plt.ylabel("Actual")
3 plt.xlabel("Predicted")
4 plt.tight_layout()
5 buf = io.BytesIO()
6 plt.savefig(buf, format="png")
7 plt.close()
8 buf.seek(0)
9 cm_base64 = base64.b64encode(buf.getvalue()).decode("utf-8")

```

Membuat grafik confusion matrix dan menyimpannya dalam format base64 agar bisa ditampilkan di halaman HTML.

```

1 train_links = build_export_links(
2     export_base, "train", features, freq_tables_train, likelihoods_train_raw, prior_table_train_df
3 )
4 full_links = build_export_links(
5     export_base, "full", features, freq_tables_full, likelihoods_full_raw, prior_table_full_df
6 )
7 split_links = build_split_links(export_base, X_train, y_train, X_test, y_test, y_pred, target_col)

```

Hasil seperti *frequency table*, *likelihood table*, *prior*, *data split*, dan *prediksi* disimpan ke file .csv agar bisa diunduh.

```

1  return render_template(
2      "result.html",
3      filename=filename,
4      head=df.to_html(classes="table table-striped", index=False),
5
6      # Split view + unduh
7      x_train=X_train.to_html(classes="table table-bordered table-sm", index=False),
8      y_train=y_train.to_frame(name=target_col).to_html(classes="table table-bordered table-sm", index=False),
9      x_test=X_test.to_html(classes="table table-bordered table-sm", index=False),
10     y_test=y_test.to_frame(name=target_col).to_html(classes="table table-bordered table-sm", index=False),
11     y_pred=pd.DataFrame({"Predicted": y_pred}).to_html(classes="table table-bordered table-sm", index=False),
12
13     # TRAIN
14     prior_table_train_html=prior_table_train_html,
15     freq_tables_train=[c: t.to_html(classes="table table-bordered table-sm", index=False, border=0)
16                        for c, t in freq_tables_train.items()],
17     likelihoods_train=[c: t.round(3).to_html(classes="table table-bordered table-sm", index=False, border=0)
18                      for c, t in likelihoods_train_html.items()],
19     train_size=len(X_train),
20
21     # FULL
22     prior_table_full_html=prior_table_full_html,
23     freq_tables_full=[c: t.to_html(classes="table table-bordered table-sm", index=False, border=0)
24                      for c, t in freq_tables_full.items()],
25     likelihoods_full=[c: t.round(3).to_html(classes="table table-bordered table-sm", index=False, border=0)
26                     for c, t in likelihoods_full_html.items()],
27     full_size=len(X_full),
28
29     # links
30     download_train=train_links,
31     download_full=full_links,
32     download_split=split_links,
33
34     accuracy=accuracy,
35     report=report,
36     cm_base64=cm_base64
37 )

```

Menampilkan semua hasil perhitungan, tabel, grafik, dan link unduh ke halaman result.html.

```

1  @app.route("/predict", methods=["GET", "POST"])
2  def predict():
3      global current_state
4      if not current_state:
5          return redirect(url_for("index"))
6      features = current_state["features"]
7      if request.method == "POST":
8          input_data = {f: request.form[f] for f in features}
9          pred = current_state["predict_row"](input_data)
10         return render_template("predict.html", features=features, result=pred)

```

Menampilkan form input fitur agar pengguna bisa mencoba prediksi manual menggunakan model yang sudah dilatih.

```

1  @app.route("/exports/<path:subpath>")
2  def download_export(subpath):
3      export_root = os.path.join(app.config["UPLOAD_FOLDER"], "exports")
4      abs_path = os.path.join(export_root, subpath)
5      directory = os.path.dirname(abs_path)
6      filename = os.path.basename(abs_path)
7      return send_from_directory(directory, filename, as_attachment=True)

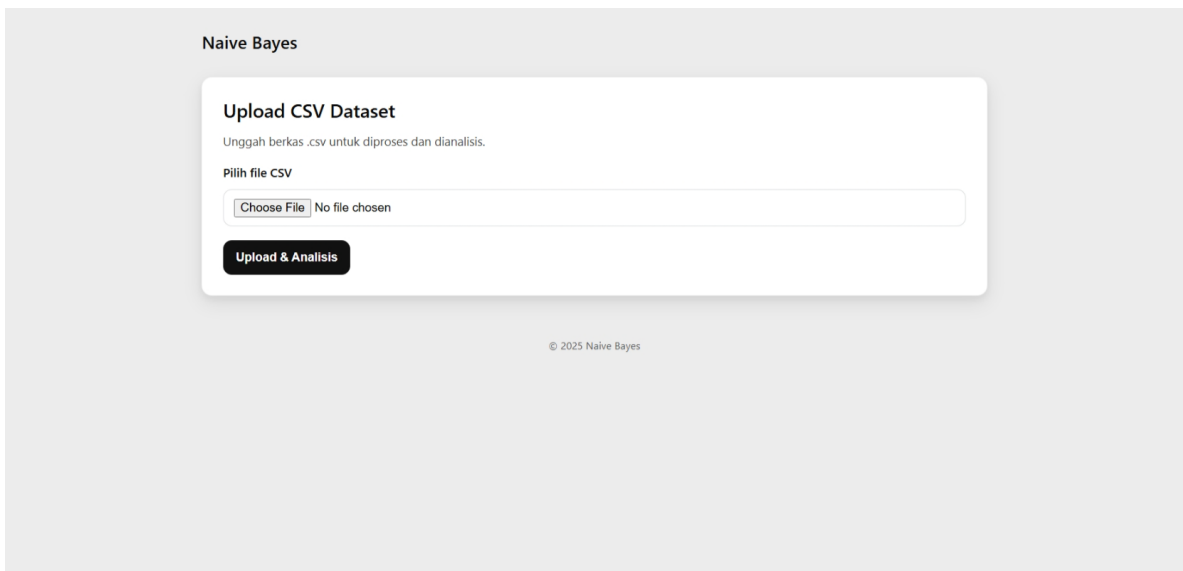
```

Menyediakan link download file hasil analisis (CSV) dari folder uploads/exports.



Menjalankan aplikasi Flask dalam mode debug agar mudah dipantau selama pengembangan.

2.5 Output UI



Naive Bayes

Hasil Analisis Dataset: [car_evaluation.csv](#)

Dataset

buying	maint	doors	persons	lug_boot	safety	class
vhigh	vhigh	2	2	small	low	unacc
vhigh	vhigh	2	2	small	med	unacc
vhigh	vhigh	2	2	small	high	unacc
vhigh	vhigh	2	2	med	low	unacc
vhigh	vhigh	2	2	med	med	unacc
vhigh	vhigh	2	2	med	high	unacc
vhigh	vhigh	2	2	big	low	unacc
vhigh	vhigh	2	2	big	med	unacc
vhigh	vhigh	2	2	big	high	unacc

Data Training (80%)

x_train

[unduh CSV](#)

buying	maint	doors	persons	lug_boot	safety
med	med	3	4	med	med
med	high	3	2	small	med
low	high	5more	more	small	med
med	low	2	4	med	high
vhigh	low	3	2	small	high
high	high	2	4	small	med
med	vhigh	5more	4	small	low
med	low	5more	4	med	med
high	med	3	4	med	low

y_train

[unduh CSV](#)

class
acc
unacc
acc
good
unacc
unacc
unacc
good
unacc

Data Testing (20%)

x_test

[unduh CSV](#)

buying	maint	doors	persons	lug_boot	safety
low	low	4	more	med	med
med	vhigh	3	4	big	low
high	vhigh	5more	4	small	high
med	med	2	2	small	low
high	high	3	more	big	high
vhigh	high	4	2	big	med
med	vhigh	4	4	small	med
vhigh	low	4	4	small	high
low	high	5more	4	big	low

y_test

[unduh CSV](#)

class
good
unacc
unacc
unacc
acc
unacc
unacc
acc
unacc

y_pred

[unduh CSV](#)

Predicted
good
unacc
unacc
unacc
acc
unacc
unacc
unacc

Probabilitas Prior — Data Training (N=1382)

Class	Count	Prior
unacc	968	0.700434
acc	307	0.222142
good	55	0.039797
vgood	52	0.037627

[unduh CSV](#)

Tabel Frekuensi — Data Training (feature × class)

► **buying** [unduh CSV](#)

► **maint** [unduh CSV](#)

► **doors** [unduh CSV](#)

► **persons** [unduh CSV](#)

► **lug_boot** [unduh CSV](#)

► **safety** [unduh CSV](#)

Likelihood — Data Training ($P(\text{feature}=\text{value} \mid \text{class})$)

► **buying** [unduh CSV](#)

► **maint** [unduh CSV](#)

► **doors** [unduh CSV](#)

► **persons** [unduh CSV](#)

► **lug_boot** [unduh CSV](#)

► **safety** [unduh CSV](#)

Probabilitas Prior — Dataset (N=1728)

Class	Count	Prior
unacc	1210	0.700231
acc	384	0.222222
good	69	0.039931
vgood	65	0.037616

[unduh CSV](#)

Tabel Frekuensi — Dataset (feature × class)

► buying	unduh CSV
► maint	unduh CSV
► doors	unduh CSV
► persons	unduh CSV
► lug_boot	unduh CSV
► safety	unduh CSV

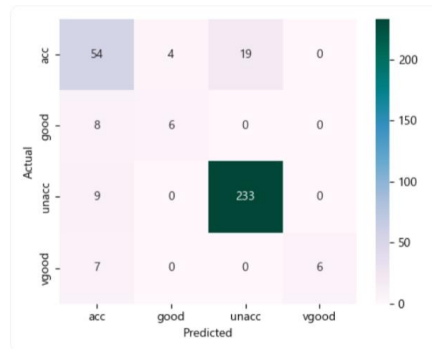
Likelihood — Dataset ($P(\text{feature}=\text{value} \mid \text{class})$)

► buying	unduh CSV
► maint	unduh CSV
► doors	unduh CSV
► persons	unduh CSV
► lug_boot	unduh CSV
► safety	unduh CSV

Classification Report

	precision	recall	f1-score	support
acc	0.69	0.70	0.70	77
good	0.60	0.43	0.50	14
unacc	0.92	0.96	0.94	242
vgood	1.00	0.46	0.63	13
accuracy			0.86	346
macro avg	0.80	0.64	0.69	346
weighted avg	0.86	0.86	0.86	346

Confusion Matrix (Visual)



Akurasi Model: 86.42%

Coba Prediksi Manual

© 2025 Naive Bayes

Naive Bayes

Prediksi Manual

buying

vhigh

maint

low

doors

4

persons

4

lug_boot

small

safety

high

Prediksi

Kembali ke Upload

© 2025 Naive Bayes

Hail dan pembahasan Output:

- Aplikasi berhasil membaca dataset dan memisahkan data training serta testing dengan perbandingan 80:20.

- Probabilitas prior ditampilkan untuk setiap kelas sebagai peluang awal sebelum memperhitungkan fitur.
- Sistem menampilkan tabel frekuensi dan tabel likelihood untuk setiap fitur.
- Hasil evaluasi berupa akurasi model, classification report, dan confusion matrix divisualisasikan dengan jelas.
- Fitur prediksi manual dapat digunakan untuk menguji hasil prediksi terhadap data baru.
- Semua hasil analisis dapat diunduh dalam format CSV melalui tautan unduhan yang tersedia.

2.6 Perhitungan Manual

PERHITUNGAN MANUAL									
PER FITUR					PER TESTING				
Tabel Frekuensi buying					Tabel Frekuensi doors				
buying	acc	good	unacc	good Total	doors	acc	good	unacc	good Total
low	30	0	259	0	low	0	0	0	0
high	55	26	201	31	high	0	0	0	0
med	25	15	139	41	med	0	0	0	0
think	55	0	259	0	think	0	0	0	0
Grand Total	307	55	660	52	Grand Total	307	55	660	52
Tabel Likelihood buying					Tabel Likelihood doors				
buying	acc	good	unacc	good	doors	acc	good	unacc	good
low	0,097	0,000	0,393	0,000	low	0,000	0,000	0,000	0,000
high	0,179	0,473	0,305	0,058	high	0,000	0,000	0,000	0,000
med	0,081	0,273	0,211	0,077	med	0,000	0,000	0,000	0,000
think	0,179	0,000	0,393	0,000	think	0,000	0,000	0,000	0,000
Grand Total	0,307	0,727	0,305	0,058	Grand Total	0,000	0,000	0,000	0,000
Tabel Frekuensi maint					Tabel Frekuensi persons				
maint	acc	good	unacc	good Total	persons	acc	good	unacc	good Total
low	0	0	259	0	low	0	0	0	0
high	0	0	259	0	high	0	0	0	0
med	0	0	259	0	med	0	0	0	0
think	0	0	259	0	think	0	0	0	0
Grand Total	0	0	1018	0	Grand Total	0	0	0	0
Tabel Likelihood maint					Tabel Frekuensi lug_boot				
maint	acc	good	unacc	good	lug_boot	acc	good	unacc	good Total
low	0,000	0,000	0,259	0,000	low	0	0	0	0
high	0,000	0,000	0,259	0,000	high	0	0	0	0
med	0,000	0,000	0,259	0,000	med	0	0	0	0
think	0,000	0,000	0,259	0,000	think	0	0	0	0
Grand Total	0,000	0,000	1,036	0,000	Grand Total	0	0	0	0
Tabel Frekuensi doors					Tabel Likelihood safety				
doors	acc	good	unacc	good Total	safety	acc	good	unacc	good Total
2	12	14	252	16	low	0	0	0	0
3	15	15	244	16	high	0	0	0	0
4	14	14	234	14	med	0	0	0	0
5more	0	0	259	0	think	0	0	0	0
Grand Total	307	55	660	52	Grand Total	0	0	0	0
Tabel Likelihood doors					Tabel Frekuensi persons				
doors	acc	good	unacc	good	persons	acc	good	unacc	good Total
2	0,039	0,255	0,385	0,039	low	0	0	0	0
3	0,048	0,273	0,385	0,048	high	0	0	0	0
4	0,045	0,255	0,385	0,045	med	0	0	0	0
5more	0,000	0,000	0,393	0,000	think	0	0	0	0
Grand Total	0,132	0,783	0,393	0,132	Grand Total	0	0	0	0
Tabel Frekuensi persons					Tabel Likelihood lug_boot				
persons	acc	good	unacc	good Total	lug_boot	acc	good	unacc	good Total
2	0	0	259	0	low	0	0	0	0
3	0	0	259	0	high	0	0	0	0
4	0	0	259	0	med	0	0	0	0
5more	0	0	259	0	think	0	0	0	0
Grand Total	0	0	1036	0	Grand Total	0	0	0	0
Tabel Likelihood persons					Tabel Frekuensi safety				
persons	acc	good	unacc	good	safety	acc	good	unacc	good Total
2	0,000	0,000	0,259	0,000	low	0	0	0	0
3	0,000	0,000	0,259	0,000	high	0	0	0	0
4	0,000	0,000	0,259	0,000	med	0	0	0	0
5more	0,000	0,000	0,259	0,000	think	0	0	0	0
Grand Total	0,000	0,000	1,036	0,000	Grand Total	0	0	0	0
Tabel Frekuensi lug_boot					Tabel Likelihood safety				
lug_boot	acc	good	unacc	good Total	safety	acc	good	unacc	good Total
low	0	0	259	0	low	0	0	0	0
high	0	0	259	0	high	0	0	0	0
med	0	0	259	0	med	0	0	0	0
think	0	0	259	0	think	0	0	0	0
Grand Total	0	0	1036	0	Grand Total	0	0	0	0
Tabel Likelihood lug_boot					Tabel Frekuensi safety				
lug_boot	acc	good	unacc	good	safety	acc	good	unacc	good Total
low	0,000	0,000	0,259	0,000	low	0	0	0	0
high	0,000	0,000	0,259	0,000	high	0	0	0	0
med	0,000	0,000	0,259	0,000	med	0	0	0	0
think	0,000	0,000	0,259	0,000	think	0	0	0	0
Grand Total	0,000	0,000	1,036	0,000	Grand Total	0	0	0	0
Tabel Frekuensi safety					Tabel Likelihood safety				
safety	acc	good	unacc	good Total	safety	acc	good	unacc	good Total
low	0	0	259	0	low	0	0	0	0
high	0	0	259	0	high	0	0	0	0
med	0	0	259	0	med	0	0	0	0
think	0	0	259	0	think	0	0	0	0
Grand Total	0	0	1036	0	Grand Total	0	0	0	0
Tabel Likelihood safety					Tabel Frekuensi safety				
safety	acc	good	unacc	good	safety	acc	good	unacc	good Total
low	0,000	0,000	0,259	0,000	low	0	0	0	0
high	0,000	0,000	0,259	0,000	high	0	0	0	0
med	0,000	0,000	0,259	0,000	med	0	0	0	0
think	0,000	0,000	0,259	0,000	think	0	0	0	0
Grand Total	0,000	0,000	1,036	0,000	Grand Total	0	0	0	0

(lebih jelasnya bisa di cek di spreadsheet bagian sheet Perhitungan manual yang sudah saya lampirkan dalam zip ini)

Prior kelas

Diambil dari proporsi masing-masing kelas pada data latih (total 1382):

- $\text{acc} = 307/1382 \approx \mathbf{0,222}$
- $\text{good} = 55/1382 \approx \mathbf{0,040}$
- $\text{unacc} = 968/1382 \approx \mathbf{0,700}$
- $\text{vgood} = 52/1382 \approx \mathbf{0,038}$

Likelihood per fitur ($P(x_i|\text{kelas})$)

Untuk tiap fitur (buying, maint, doors, persons, lug_boot, safety) kamu buat **tabel frekuensi** per kelas, lalu dibagi jumlah per kelas menjadi **tabel likelihood**.

- $P(\text{buying}=\text{low} \mid \text{good}) = \mathbf{0,655}$
- $P(\text{maint}=\text{low} \mid \text{good}) = \mathbf{0,655}$
- $P(\text{doors}=4 \mid \text{good}) = \mathbf{0,255}$
- $P(\text{persons}=\text{more} \mid \text{good}) = \mathbf{0,509}$

- $P(\text{lug_boot}=\text{med} \mid \text{good}) = \mathbf{0,364}$
- $P(\text{safety}=\text{med} \mid \text{good}) = \mathbf{0,545}$

Hitung skor (posterior tak ternormalisasi)

Untuk sampel uji $x_{\text{test}} = \{\text{buying}=\text{low}, \text{maint}=\text{low}, \text{doors}=4, \text{persons}=\text{more}, \text{lug_boot}=\text{med}, \text{safety}=\text{med}\}$, skor tiap kelas = **Prior** $\times \prod$ **likelihood fitur**.

Contoh kelas *good*:

score_good

$$= 0,040 \times 0,655 \times 0,655 \times 0,255 \times 0,509 \times 0,364 \times 0,545$$

$$\approx 0,000044187$$

(Nilai lainnya di sheet: score_unacc $\approx 0,000021558$ • score_acc $\approx 0,000022095$ •

score_vgood = 0 karena ada likelihood 0.)

Normalisasi $\rightarrow$ Posterior

Posterior_kelas = score_kelas / (jumlah semua score).

Hasil di sheet:

- $P(\text{unacc} \mid x) \approx \mathbf{0,2454}$
- $P(\text{acc} \mid x) \approx \mathbf{0,2515}$
- $P(\text{good} \mid x) \approx \mathbf{0,5030}$
- $P(\text{vgood} \mid x) \approx \mathbf{0,0000}$

Prediksi

Ambil **posterior terbesar** adalah **prediksi = good**.

Link GitHub:

[Naive Bayes Web](#)